

Cognizant – Module 1

Design Patterns and Principles

The Reason for SOLID Principles

- The SOLID principles were introduced by Robert C. Martin. These five principles have revolutionized the world of object-oriented programming, changing the way that we write software.
- Martin and Feathers' design principles encourage us to create more maintainable, understandable, and flexible software.
- As our applications grow in size, we can reduce their complexity, which can save us a lot of time and headache.

The five concepts of SOLID principles are:

1. Single Responsibility Principle (SRP)
2. Open/Closed Principle (OCP)
3. Liskov Substitution Principle (LSP)
4. Interface Segregation Principle (ISP)
5. Dependency Inversion Principle (DIP)

1. Single Responsibility Principle (SRP)

It states that a class should have only one responsibility, and it should also have only one reason to change.

Benefits

6. Testing — A class with one responsibility will have far fewer test cases.
7. Lower coupling — Less functionality in a single class will have fewer dependencies.
8. Organization — Smaller, well-organized classes are easier to search than monolithic ones.

Example: Book Class

Step 1 — If we take a partial Book class, this class will be responsible for storing book data, i.e. book name, author, text content. So far, it has one responsibility → managing book information.

Step 2 — Adding Text-Related Methods: these methods work with the book's text. Responsibility is still related to the book itself, so SRP is not violated.

Step 3 — Adding Printing Functionality: now the Book class has two jobs — (1) manage book data, (2) print book data. Now SRP is violated.

Fixing with BookPrinter

Now responsibilities are separated:

- Book class → store / manage book data
- BookPrinter class → print text

Final structure:

```
Book book = new Book(...);
BookPrinter printer = new BookPrinter();
printer.printTextToConsole(book.getText());
```

2. Open for Extension, Closed for Modification (OCP)

Software entities (classes, methods, modules) should be open for extension but closed for modification.

We are able to add new functionality, but after testing, we are unable to change tested code.

Example: Guitar Class

```
public class Guitar {
    private String make;
    private String model;
    private int volume;
}
```

This class is working perfectly.

Requirement Change

Client says: "I want a guitar with flames painted on it." If we edit the existing class for this, OCP will be violated.

Following OCP — instead of modifying Guitar, create a new class:

```
public class SuperCoolGuitarWithFlames extends Guitar {
    private String flameColor;
}
```

The Guitar class remains unchanged. Now we can create:

```
SuperCoolGuitarWithFlames coolGuitar = new SuperCoolGuitarWithFlames();
```

Now it is closed for modification and open for extension.

3. Liskov Substitution Principle (LSP)

If class A is a subtype (child class) of class B, then we should be able to use A wherever B is expected, without breaking the program.

Example: Car Interface

```
public interface Car {
    void turnOnEngine();
    void accelerate();
}
```

Every car must turn on engine and accelerate.

If we take a motor car, it will work perfectly because a petrol car has an engine and acceleration. But when we take an electric car, it does not have a traditional engine — then the program fails. LSP is violated here.

To solve this, we have to create different interfaces, like Common Car Interface, Engine Car Interface, Motor Car / Electric Car Interfaces.

A Small Example

- Bird class (fly()) — parent → Sparrow (child class) ✓ follows

- Bird class (fly()) — parent → Penguin (child) ✗ does not follow

4. Interface Segregation Principle (ISP)

ISP states that no class should be forced to implement methods it does not use.

Example Problem: Large Interface

```
public interface BearKeeper {  
    void washTheBear();  
    void feedTheBear();  
    void petTheBear();  
}
```

This interface has 3 jobs — bathing, feeding, and petting the pet. But if one person is a cleaner only, then he/she should go to all the methods mentioned in the interface.

To avoid this, we will create small interfaces, like:

9. Cleaning Interface
10. Feeding Interface
11. Petting Interface

BearCarer Class

```
public class BearCarer implements BearCleaner, BearFeeder {  
    public void washTheBear() { }  
    public void feedTheBear() { }  
}
```

Now it will work properly — the user can write code repeatedly to write code only.

5. Dependency Inversion Principle (DIP)

High level modules should not depend on low-level modules. They should depend on abstractions (interfaces).

Example — Without DIP

Assuming we have one computer:

```
class Windows98Machine {  
    private StandardKeyboard keyboard;  
    public Windows98Machine() {  
        keyboard = new StandardKeyboard();  
    }  
}
```

A Windows 98 machine is built with a standard keyboard on its own. But the client says they need a gaming keyboard. Then we have to modify the Windows 98 machine — existing code changes.

With DIP (Solution)

Create an interface first:

```
interface Keyboard { }
```

```
class StandardKeyboard implements Keyboard { }
```

```
class GamingKeyboard implements Keyboard { }
```

→ No modification needed; easy testing.

Design Patterns

A Design pattern is a reusable template or best practice used to solve common software design problems.

Why Use Design Patterns?

12. Make code reusable
13. Easy to maintain
14. Scalable
15. Follows best practices
16. Provide a common language between developers

Gang of Four (GoF)

A group of 4 authors that made design patterns popular.

Names: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides.

In 1994 they wrote a book — “Design Patterns: Elements of Reusable Object-Oriented Software”.

GoF Design Pattern Categories

17. Creational Patterns (object creation)
18. Structural Patterns (object structure)
19. Behavioral Patterns (object behavior)
20. Architectural Patterns

GoF gave 23 design patterns which come under the 3 categories above. The 4th — Architectural — does not come under GoF.

1. Creational Patterns

These patterns focus on object creation mechanisms, providing ways to create objects in a manner that is flexible, decoupled from their concrete implementations.

Examples

1. Singleton Pattern

- It allows only one object to be created from one class.
- Real life example — in a bank database connection, all applications use only one database connection manager.

2. Factory Method Pattern

- It gives an interface to create an object, but which object will be created is decided by subclasses.

- Real life example — customer asks to create BMW or Audi; the factory creates the appropriate car.

3. Builder Pattern

- Constructs a complex object step-by-step.
- Real life example — add cheese, mayo, cheese, onion to a burger.

2. Structural Patterns

These patterns deal with the composition of classes and objects, forming larger structures while keeping them flexible and efficient. They help ensure that classes and objects can work together effectively to achieve a common goal.

Examples

1. Adapter Pattern

- Makes two incompatible interfaces compatible.
- Real life example — a mobile charger adapter; Indian plug used in a US socket.

2. Decorator Pattern

- Adds new functionality at runtime to an object.
- Real life example — coffee + milk, sugar, chocolate.

3. Proxy Pattern

- Controls access to the original object.
- Real life example — an ATM is not a direct bank; ATM = Proxy.

3. Behavioral Patterns

These patterns focus on the interactions and communication between objects and classes. They provide solutions for effectively managing the flow of control, responsibilities, and behavior between objects.

Examples

1. Observer Pattern

- Notifies all the dependents automatically if an object changes.
- Real life example — YouTube channel: subscribers = observer, channel = subject. Video uploaded → notify all.

2. Strategy Pattern

- Stores multiple algorithms in separate classes and chooses one at runtime.
- Real life example — Google Maps; travel by car, bike, or walking changes the algorithm.

3. Command Pattern

- Encapsulates a request in the form of an object.
- Real life example — TV remote; button press = command, TV executes.

4. Architectural Pattern

MVC (Model View Controller)

Separates the application logic into three interconnected components: the Model (data and business logic), the View (user interface), and the Controller (handles user input and updates the Model and View).

MVVM (Model-View-ViewModel) Pattern

- A variation of the MVC pattern that separates the user interface (View) from the business logic and data (Model) by introducing a new ViewModel.
- It acts as an intermediary between the View and the Model.

Dependency Injection (DI)

Creating dependencies outside the object — dependencies are created and passed in from an outer object, rather than the object creating them itself.